

第四次课后作业（图+图应用）

题量: 1 满分: 100

智能分析

100分

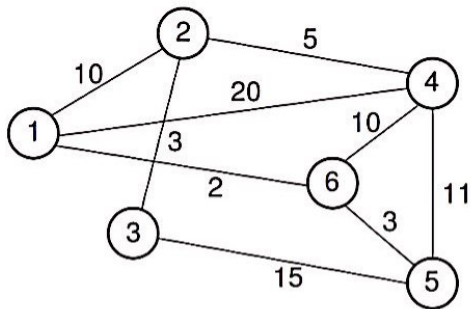
作答时间: 2025-04-02 21:31 至 2025-05-09 23:00

一. 简答题（共1题，100分）

1. (简答题)

AI讲解

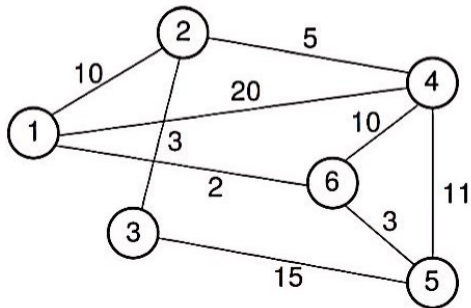
1. 【应用题】



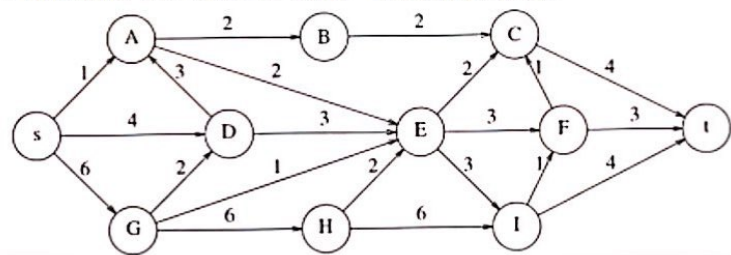
- (a) 画出所示图的相邻矩阵表示，并说明如何在相邻矩阵中计算无向图中某个顶点（该顶点在顶点表中的索引为 i）的度。
- (b) 画出所示图的邻接表表示
- (c) 设某无向图的顶点数为 n ，边数为 e ，试分别说明使用邻接矩阵和邻接表来存储该图的空间复杂度。

2. 【应用题】对于所示图，

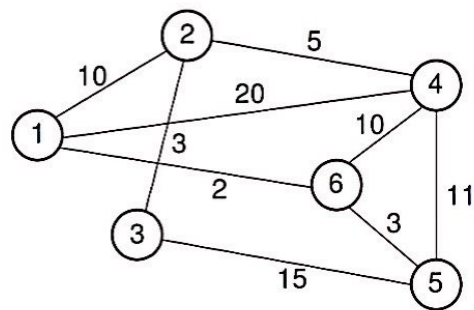
- (1) 给出从顶点1开始的DFS树和DFS遍历序列。
- (2) 给出从顶点1开始的BFS树和BFS遍历序列。



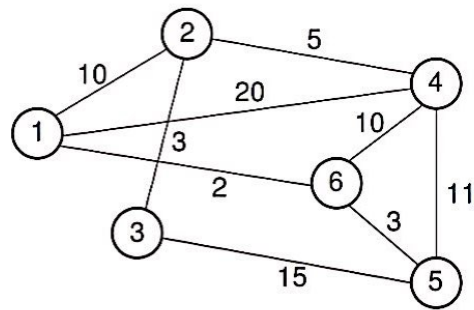
3.【应用题】对于所示图，给出一个拓扑排序序列。



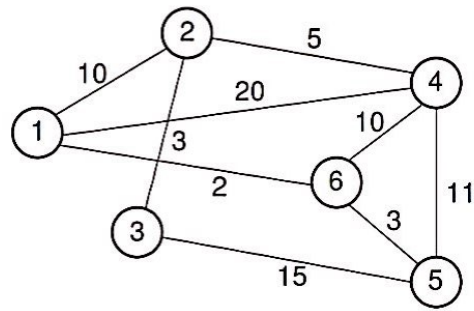
4.【应用题】对于所示图，给出从顶点4出发，使用Dijkstra最短路径算法产生的最短路径，要求参照教材271页写出过程。



5.【应用题】对于所示图，给出从顶点1出发使用Prim的MST算法时各个边的访问顺序，并给出最终的MST。



6.【应用题】对于所示图，给出使用Kruskal的MST算法时各个边的访问顺序，描述依次将权值最小的剩余边加入到MST的过程（如教材图8-10）。



7.【算法设计题】试设计一个算法Mark标记一个无向图G的各个连通分量，即第i个连通分量标记为i，并分行输出各个连通分量顶点序号。

- (1) 用自然语言描述算法思想；
- (2) 用类C或C++语言描述算法的实现；
- (3) 分析算法的时间复杂度。

注意：课后作业必须在作业本上完成，需要有完整的计算过程。在截止时间之前，在系统提交答案照片。

1.

(a) 邻接矩阵

	1	2	3	4	5	6
1	0	10	∞	20	∞	2
2	10	0	3	5	∞	∞
3	∞	3	0	∞	15	∞
4	20	5	∞	0	11	10
5	∞	∞	15	11	0	3
6	2	∞	∞	10	3	0

如何计算 V_i 的度:

计算这个顶点 v_i 在邻接矩阵中第 i 行 (或第 i 列) 非 0 非 ∞ 元素个数即为度

(b) 邻接表

es.

data	firstedge	adjrx	weight	next

1 → 2/10 → 4/20 → 6/2/1

2 → 1/10 → 3/2 → 4/5/1

3 → 2/3 → 5/10/1

4 → 1/20 → 2/5 → 5/11 → 6/10/1

5 → 3/15 → 4/11 → 6/3/1

6 → 1/2 → 4/10 → 5/3/1

(3) 顶点数为 n , 边数为 e

空间复杂度

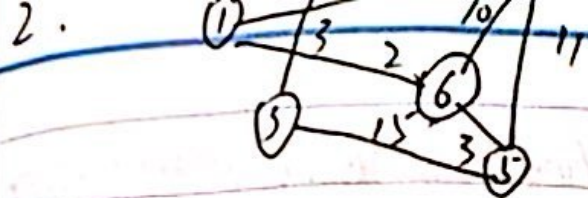
邻接矩阵: $O(n^2)$

共有 n 个顶点, 邻接矩阵需要存储一个大小为 $n \times n$ 的二维数组, 故空间复杂度为 $O(n^2)$

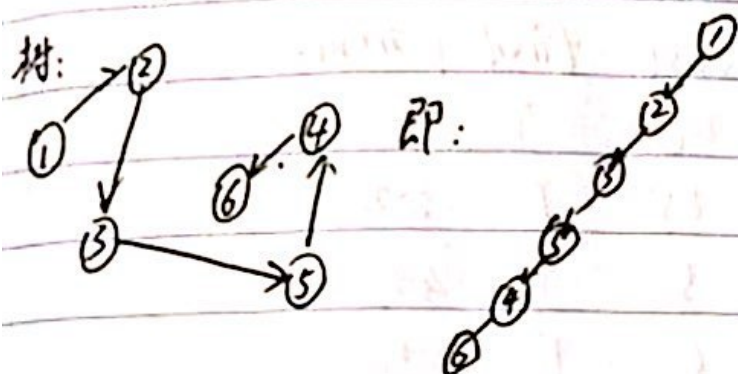
邻接表: $O(n+e)$.

每个顶点需要存储一个链表, 共需 n 个链表.

每条边会被存储两次, 一次在一个顶点的链表中, 一次在另一个顶点的链表中, 存储 2 条边
故空间复杂度为 $O(n+2e)$.

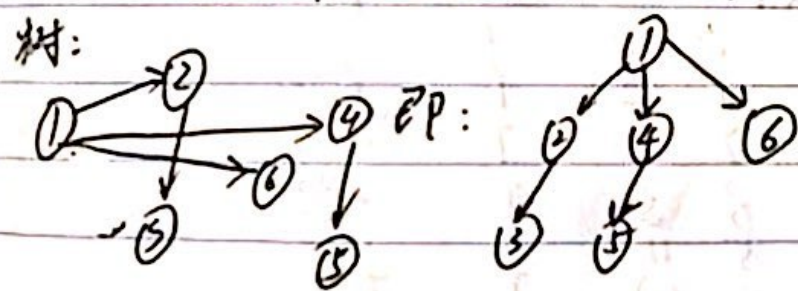


(1) 从顶点 1 开始的 DFS 树和 DFS 遍历序列



序列: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$

(2) 从顶点 1 开始的 BFS 树和 BFS 序列



序列: $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 3 \rightarrow 5$

3. 给出个拓扑排序

答: S, G, D, A, B, H, E, I, F, C, t.

① 从 S 开始, 入度为 0, 输出 S.

② G 入度为 0, 输出 G.

③ D 入度为 0, 输出 D.

④ A 入度为 0, 输出 A.

⑤ B 入度为 0, 输出 B.

⑥ H 入度为 0, 输出 H. ⑦ E 入度为 0 输出 E ⑧ I 入度为 0 输出 I

⑨ F 入度为 0 输出 F ⑩ C 入度为 0 输出 C ⑪ t 入度为 0 输出 t.

故拓扑排序为 S, G, D, A, B, H, E, I, F, C, t.

4. Dijkstra算法产生的最短路径.

先写过程,再画课本表格. dist为路径长度. final为是否遍历完成 parent为前驱节点. 为简便表示, T为True, F为False.

①: 从4开始

② 2最小, 从2开始

	dist	final	parent
4	0	T	
1	20	F	4→1
2	5	F	4→2
3	∞	F	-1
5	11	F	4→5
6	10	F	4→6

	dist	final	parent
4	0	T	
1	15	F	2→1
2	5	T	4→2
3	8	F	2→3
5	11	F	4→5
6	10	F	4→6

③ 3最小

④ 6最小

	dist	final	parent
4	0	T	
1	15	F	2→1
2	5	T	4→2
3	8	T	2→3
5	11	F	4→5
6	10	F	4→6

	dist	final	parent
4	0	T	
1	12	F	6→1
2	5	T	4→2
3	8	T	2→3
5	11	F	4→5
6	10	T	4→6

⑤ 5最小

⑥ 1最小, 此时遍历完成! 得到最短路径

	dist	final	parent
4	0	T	
1	12	F	6→1
2	5	T	4→2
3	8	T	2→3
5	11	T	4→5
6	10	T	4→6

	dist	final	parent
4	0	T	
1	12	T	6→1
2	5	T	4→2
3	8	T	2→3
5	11	T	4→5
6	10	T	4→6

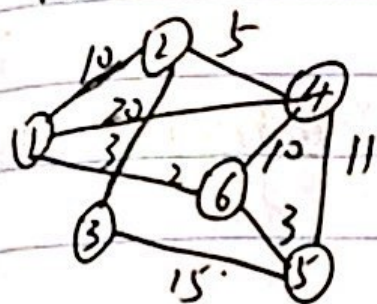
最短路径为: 1) 顶点 2) 路径 3) 路径长度

顶点	路径	路径长度
1	4→6→1	12
2	4→2	5
3	4→2→3	8
5	4→5	11
6	4→6	10

顶点	dist	final	parent
1	20	F	4→1
2	5	F	4→2
3	8	F	2→3
5	11	F	4→5
6	10	F	4→6

5 {4,2,5,6} {4,2,3,5,6} {4,1,2,3,5,6}

5. Prim的MST. 与各顶点访问顺序



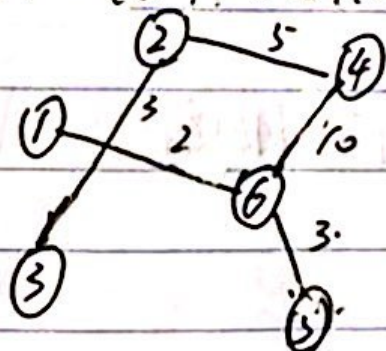
- ① 从1出发: (1,6)权值为2最短. 访问 (1,6)
- ② 顶点为1,6的边中 (6,5)权值为3最短. 访问 (6,5)
- ③ 1,5,6为顶点边中 (6,4)权值为10最小. 访问 (6,4)
- ④ 1,4,5,6为顶点边中 (4,2)权值为5最小. 访问 (4,2)
- ⑤ 1,2,4,5,6顶点边中 (2,3)权值为3最小. 访问 (2,3).

此时访问5次为 $(V-1)$ 次, 访问结束!

故访问顺序为

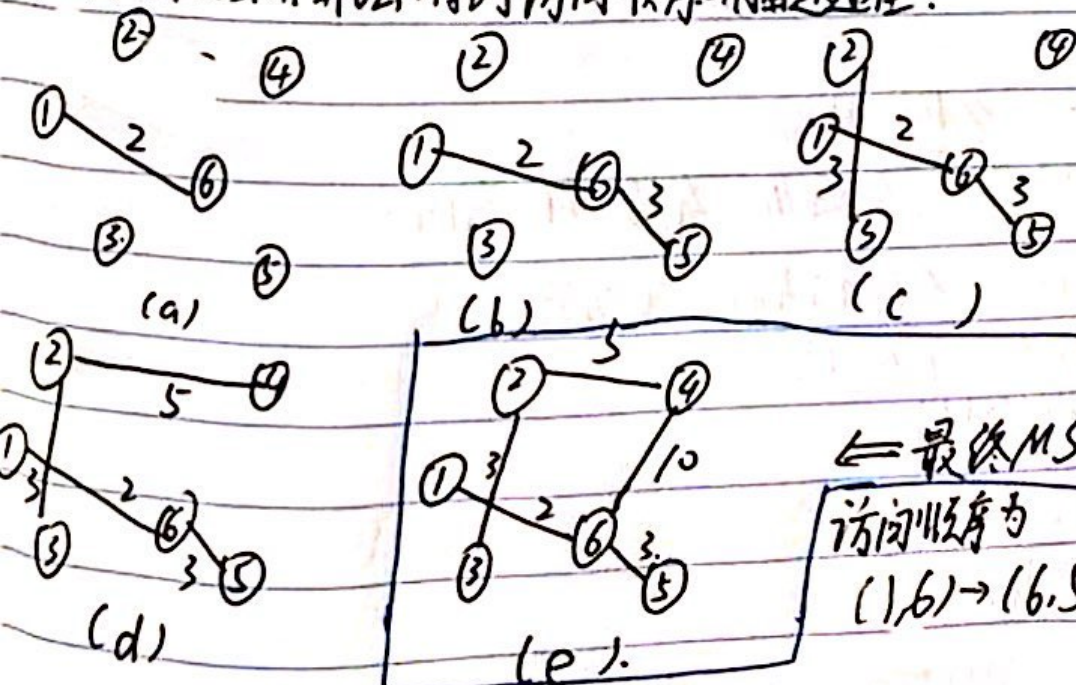
① $(1,6) \rightarrow (6,5) \rightarrow (6,4) \rightarrow (4,2) \rightarrow (2,3)$.

② MST 为:



pair	weight
(1,6)	2
(6,5)	3
(2,3)	3
(2,4)	5
(4,6)	10
(1,2)	10
(4,5)	11
(3,5)	15

6. 使用kruskal算法, 得到访问顺序并描述过程.



← 最终MST.

访问顺序为

$(1,6) \rightarrow (6,5) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (4,6)$

7 (1) 算法设计思想

使用DFS标记一个无向图G的每个连通分量

① 初始化一个标记数组, 记录每个结点是否被访问过并存储当前连通分量

② 遍历所有结点, 对于每个未访问的结点, 从该结点开始DFS搜索, 标记与该结点的节点并放入连通分量列表中

③ 输出连通分量, 每完成一次DFS输出该连通分量顶点序号

④ 重复该过程, 直至所有结点都被访问

(2) 代码

```
void DFS (int u, vector<vector<int>> & graph, vector<bool> & visited, vector<int> & component,
```

```
{ stack<int> s; // 存储连通分量
```

```
    s.push(u);
```

```
    visited[u] = true;
```

```
    while (!s.empty()) {
```

```
        int node = s.top();
```

```
        s.pop();
```

```
        component.push_back(node); // 添加当前节点到连通分量
```

```
        for (int v: graph[node]) {
```

```
            if (!visited[v]) {
```

```
                visited[v] = true;
```

```
                s.push(v);
```

```
            }
```

```
        }
```

```
    }
```

// DFS代码.

```
for (int i=0; i<n; i++) {
```

```
    if (!visited[i]) {
```

```
        vector<int> component; // 存储当前连通分量
```

```
        DFS(i, graph, visited, component);
```

```
        for (int vertex: component) {
```

```
            cout << vertex << " ";
```

```
        }
```

```
        cout << endl;
```

(3) 时间复杂度: $O(m+n)$.

每条边和每个结点都DFS一次, 边的数量为 m , 顶点数量为 n

故时间复杂度为 $O(m+n)$.